

TECHNICAL ARCHITECTURE DOCUMENT

FacilityPro CMMS New Web Application Architecture

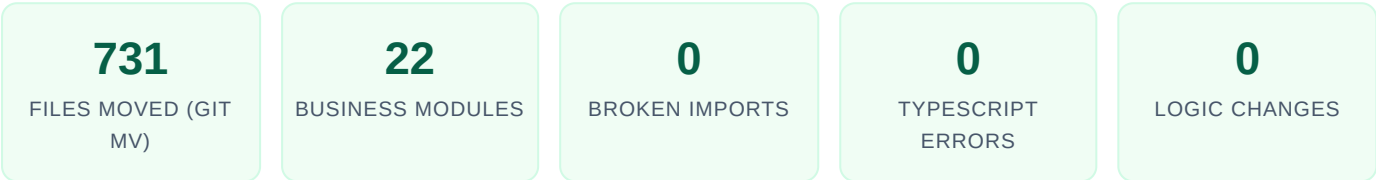
Enterprise-grade, feature-based modular architecture — Next.js 16 ·
TypeScript · Prisma

Project	FacilityPro — Facility Management & CMMS Platform
Repository	hsphotographyhasib-design/mohd_hms
Architecture rev.	Feature-Based Modular Restructure (commit f82ed88)
Date	9 July 2026
Status	Live — restructure complete, zero business-logic changes

1. Executive Summary

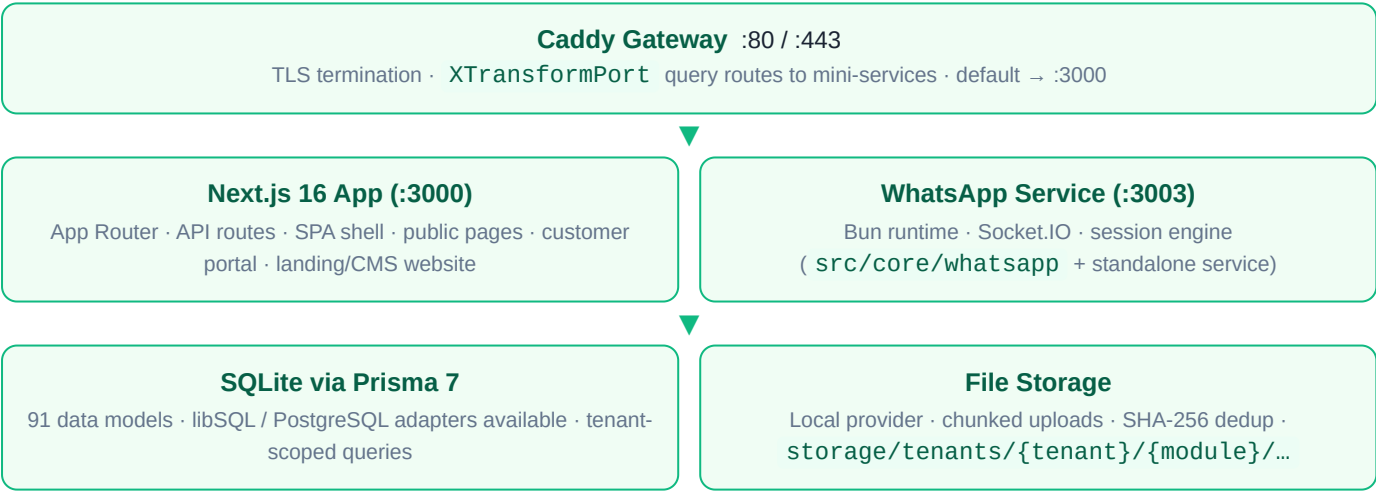
FacilityPro is a multi-tenant Computerized Maintenance Management System (CMMS) covering complaints, work orders, equipment, preventive maintenance, inventory, quotations, invoicing, finance, HR, CMS/website building, and WhatsApp-integrated communication. The codebase was restructured from a flat layout into an **enterprise-grade, feature-based modular architecture**: every business capability now lives in a self-contained module, supported by clearly separated `core`, `shared`, and application-shell layers.

The restructure preserved all functionality:



- All files relocated with `git mv` — full history preserved.
- Every module and layer exposes a **barrel export** (`index.ts`); no deep relative imports (`../..`) remain.
- All API routes kept their original URLs; UI/UX, branding and navigation are unchanged.

2. System Architecture



The web app is a **single-page application**: all authenticated views render on the `/` route and are switched client-side through a Zustand view store (section 6), while Next.js file-based routing serves only API endpoints, public pages, and the customer portal.

3. Source Layer Architecture

`src/` is organised as a layered stack. Dependencies flow strictly **downward**: application entry points consume modules; modules consume shared and core; core depends on nothing above it.

<code>src/app/</code>	Next.js App Router — API route handlers and page entries. URLs unchanged by the restructure; thin delegating layer only.	299 files
<code>src/app-shell/</code>	Authenticated SPA shell — AppShell, Sidebar, Header, floating nav, providers, login view, navigation store.	12 files
<code>src/landing/</code>	Public marketing/landing website — sections, themes, CMS-rendered pages.	38 files
<code>src/mobile-app/</code>	Mobile shell and all mobile screens (technician-focused experience).	17 files
<code>src/customer-portal/</code>	Customer-facing portal and its store.	9 files
<code>src/modules/</code>	22 self-contained business feature modules — each with its own components and barrel export (section 4).	156 files
<code>src/shared/</code>	Cross-cutting UI kit (shadcn/ui), layouts, brand components, hooks, types, utils, constants, validators.	66 files
<code>src/core/</code>	Platform foundation — auth/RBAC, database, config, storage, email, WhatsApp engine, maps, AI, workflow, errors, logger, middleware.	100 files

Directory reference

<code>src/</code>	
├─ <code>app/</code>	# Next.js App Router: <code>/api/**</code> route handlers + pages (URLs unchanged)
├─ <code>app-shell/</code>	# AppShell, sidebar, header, nav/, providers/, store.ts, login-view
├─ <code>landing/</code>	# Public website: sections, themes, CMS-rendered landing pages
├─ <code>mobile-app/</code>	# Mobile shell + screens
├─ <code>customer-portal/</code>	# Customer portal + portal store
├─ <code>modules/</code>	# 22 business modules (cms, complaints, work-orders, hr, ...)
│ └─ <code><module>/</code>	
│ └─ <code>components/</code>	# Module views: list / detail / form
│ └─ <code>index.ts</code>	# Barrel export – the module's ONLY public surface
├─ <code>shared/</code>	# ui/ (shadcn), components/brand, layouts/, hooks/, types/,
│	# utils/, constants/, validators/, services/
└─ <code>core/</code>	# auth/, database/, config/, storage/, email/, whatsapp/,
	# firebase/, maps/, ai/, workflow/, errors/, permissions/,
	# constants/, utils/, hooks/, logger/, middleware/, types/

Path alias: a single alias `@/*` → `./src/*` keeps imports absolute and stable, e.g. `import { db } from '@core/database'` or `import { ComplaintsView } from '@modules/complaints'`.

4. Business Modules (src/modules/)

Each module is a vertical slice of one business capability. Modules never import each other's internals — only barrel exports.

MODULE	RESPONSIBILITY	PRIMARY API SURFACE
dashboard	KPI overview, smart quick actions, role-aware widgets	/api/dashboard
complaints	Complaint intake, assignment, escalation, resolution	/api/complaints
work-orders	Work order lifecycle, technician assignment	/api/work-orders
equipment	Asset registry, QR code system, bulk operations	/api/equipment, /api/qr
pm	Preventive-maintenance schedules and checklists	/api/pm
inventory	Stock items, movements, enterprise item forms	/api/inventory
customers	Customer registry and customer-facing data	/api/customers, /api/customer
quotations	Quotation builder, price book, service items/packages	/api/quotations, /api/price-book
invoices	Invoicing with A4 print-optimised output	/api/invoices, /api/payments
finance	Financial overview, payment tracking	/api/finance
hr	Employees, attendance, payroll, departments (23 files)	/api/hr, /api/employees
purchases	Purchase orders and supplier flows	/api/purchases
vehicles	Fleet/vehicle management	/api/vehicles
technicians	Technician ops center and mobile APIs	/api/technicians
reports	Reporting and exports	/api/reports
cms	Website/CMS with drag-and-drop page builder (32 files)	/api/cms
whatsapp	WhatsApp inbox, campaigns, session UI	/api/whatsapp
email	Email composer and delivery UI	/api/email
notifications	In-app notification center, triggers, escalation rules	/api/notifications
documents	Document storage and management	/api/documents
sessions	Session administration	/api/sessions
settings	Tenant and system settings	/api/system, /api/setup

Module anatomy

```
src/modules/complaints/  
├─ components/      # complaint-list, complaint-detail, complaint-form ...  
└─ index.ts         # export { ComplaintList, ComplaintDetail, ... }  
  
src/modules/cms/
```

```
|— components/      # CMS admin UI (16+ components)
|— page-builder/    # Drag-and-drop page builder (dnd-kit)
└— index.ts
```

5. Core Layer (src/core/) — Platform Foundation

SUBSYSTEM	CONTENTS
auth/	JWT issue/verify (<code>auth.ts</code>), password hashing, NextAuth bridge, password reset, zod auth schemas, session components, RBAC (<code>rbac.ts</code>) and permission maps (<code>permissions.ts</code>)
database/	Prisma client singleton (<code>db</code>) — SQLite with libSQL/PostgreSQL adapters
config/	Centralised, validated environment access (<code>env.ts</code>) with production warnings
storage/	Storage provider abstraction — chunked upload, assembly, copy/move/stat, SHA-256 checksum dedup
whatsapp/	WhatsApp engine + service integration (session lifecycle, messaging)
email/	Email transport and templates
firebase/	Firebase integration (push notifications)
maps/	Maps/geolocation services, saved locations
ai/	AI helpers and integrations
workflow/	Workflow/escalation engine
errors/ · logger/	Typed error handling; level-aware logger
middleware/	Request middleware (auth/tenant resolution)
permissions/ · constants/ · utils/ · hooks/ · types/	Platform-level permission helpers, constants, utilities, hooks and types

6. Shared Layer, App Shell & State

src/shared/

- `ui/` — full shadcn/ui kit (New York style): button, card, dialog, table, chart, confirm-provider, ...
- `components/brand/` — brand/logo components
- `layouts/` — page layout primitives
- `hooks/` — `use-toast` , `use-mobile` , `use-secure-fetch`
- `types/ · utils/ · constants/ · validators/ · services/`

Zustand stores

STORE	KEY STATE
<code>useAuthStore</code>	user, token, login(), logout()
<code>useAppStore</code>	currentView, viewParams, sidebar, search
<code>useNotificationStore</code>	notifications[], unreadCount

SPA view routing (not file-based)

All authenticated views render on `/` ; `app-shell.tsx` maps the `AppView` union to lazy-loaded module views under `Suspense` :

```
// Navigate
useAppStore.getState().setView('equipment-detail', { id: equipmentId });

// AppShell renders the matching React.lazy() module view
AppShell
├─ AppHeader      (sticky: search, notifications, user menu)
├─ Sidebar        (desktop: collapsible 256↔68px · mobile: sheet drawer)
├─ FloatingNavBar (context-aware smart quick actions)
└─ ViewRouter → [lazy-loaded module view]
```

7. Security — RBAC & Multi-Tenancy

Role hierarchy (numeric weight — higher inherits broader access):

```
super_admin(100) > admin(90) > manager(80) > supervisor(70) > finance(60) > technician(50) > ...

canAccess(userRole, feature)      → feature-level access
hasMinRole(userRole, minRole)     → hierarchy check
hasPermission(userRole, roles[])  → explicit role list check
```

Every authenticated API route enforces **tenant isolation**: the tenant id is read from the JWT payload and applied to every Prisma query.

8. API Route Pattern

```
import { verifyToken } from '@core/auth';
import { db } from '@core/database';

export async function GET(request: NextRequest) {
  // 1. Verify JWT
  const token = request.headers.get('authorization')?.replace('Bearer ', '');
  const payload = verifyToken(token);
  if (!payload) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  // 2. Tenant from payload → 3. Tenant-scoped query → 4. Respond
  const data = await db.model.findMany({ where: { tenantId: payload.tenantId } });
  return NextResponse.json({ data });
}
```

Public routes (QR scans, WhatsApp webhooks, customer portal) skip JWT verification but remain tenant-scoped via URL/domain. A `/api/health` endpoint supports live diagnostics.

9. Import & Dependency Rules

✓ Do

```
import { ComplaintList }
  from '@modules/complaints';
import { db } from '@core/database';
import { env } from '@core/config';
import { Button } from '@shared/ui/button';
```

✗ Don't

```
// deep import into module internals
import x from
  '@modules/complaints/components/list';
// deep relative traversal
import y from '../../../../core/auth';
```

- Modules communicate only through **barrel exports** (`index.ts`).
- Dependency direction: `app` → `shell/portals` → `modules` → `shared` → `core`. Never upward.
- `src/app` route handlers stay thin — business logic lives in `modules/core`.

10. Data & File Storage

Database

- **Prisma 7 ORM**, **91 models**, SQLite provider
- Driver adapters for libSQL and PostgreSQL enable managed hosting (e.g. Render)
- Singleton client exported from `@/core/database`
- Seed script via `bun run prisma/seed.ts`

File storage

```
storage/
  tenants/
    {tenantId}/
      {module}/
        {folder}/
          {filename}
```

Chunked upload → assembly, copy/move/stat, SHA-256 dedup, behind a provider interface for future cloud backends.

11. Technology Stack

AREA	TECHNOLOGY
Framework	Next.js 16 (App Router, Turbopack) · React · TypeScript
State	Zustand (auth, app view, notifications, portal stores)
UI	Tailwind CSS · shadcn/ui (New York) · Radix primitives · Lucide icons · Framer Motion · Recharts · sonner toasts · dnd-kit (page builder) · MDX editor
Forms	react-hook-form + zod resolvers
Data	Prisma 7 · SQLite (libSQL/PostgreSQL adapters)
Messaging	WhatsApp Bun mini-service (:3003) with Socket.IO · Email service · Firebase push
Gateway	Caddy (:80/:443) with port-transform routing to mini-services
Runtime tooling	Bun (scripts/seed) · ESLint · Prisma CLI

12. Migration Outcome

GUARANTEE	RESULT
File history	All 731 files moved with <code>git mv</code> — history fully preserved
Imports	Updated across the entire codebase; 0 broken imports, no deep relative imports remain
Type safety	0 TypeScript errors; 0 lint errors (pre-existing 11 warnings unchanged)
API compatibility	All API routes remain at the same URLs
UX	UI, branding, colors, and floating navigation unchanged
Behaviour	Zero business-logic changes